

Sprint 2 Page

Team: Tolstoi Espino, Megan Soto, Frank Bugarin, Pamela Herrera, Anali Bailey

Business Case

Executive Summary:

The Subscription Manager App addresses the verified market problem of consumer financial leakage caused by forgotten recurring trials, unnoticed small fees (\$2–\$10/month), and intentionally complex cancellation workflows. Qualitative user research demonstrated recurring losses exceeding \$100 across two years for services that remained active.

The proposed solution centralizes active recurring charges, provides advance renewal notifications, calculates cumulative multi-year costs, and delivers actionable cancellation pathways. Delivery is planned across a 3-month timeline requiring 495 effort hours (3.00 FTE-Months) at a loaded labor cost of \$12,375.00 to \$15,000.00. Since interviews confirmed real demand and we can build it within our 3-month timeline using free test environments, we recommend a go decision.

Business Strategy Strategy-to-project chain:

Strategy: Help everyday subscription consumers eliminate and avoid paying hidden financial waste and take full control of where their money is being sent and consent to those charges and track them to verify long term costs.

Objective: Develop an application that informs the consumer about active subscriptions and projects the cumulative financial impacts. Also informs them about the cancellation process.

Justification: Our research shows that consumers lose money not just from forgotten trials but also from small recurring charges to completely go unnoticed over long periods of time with users wasting over \$100 across two years on unused services. Users struggle to cancel unwanted subscriptions as they make it a step-by-step nightmare to cancel.

Scope: The application will display all recurring subscriptions and feature long term cost calculators and provide direct pathways or guidance to bypass cancellation setbacks. We will create this application based on the sprints set by the professor.

Target audience: Our audience priorities would be homeowners who must pay for subscriptions for the whole family/house and are too busy to handle subscriptions on their own or notice small charges being stated in their account.

Investors: For this application, there would not be any specific group of people that would be interested in investing in this application. Banks could be interested in investing to offer better services to their customers as they get frequent calls from unauthorized subscriptions chargers from their customers.

Estimation Appendix

The team focused on one core piece of the app: “Recurring Charge Detection & Notification Setup”. This slice covers transaction records, spotting recurring charges and notifying the user with an alert before the next renewal date.

Estimation Methods

The slice was estimated using two different approaches:

1. Function Points: Counted the screens, inputs, outputs, files, and other actions needed to make the slice work, which came out to 44 function points. Based on an assumed benchmark range of 6 to 12 hours per point, this method estimated between 264 and 528 team hours. ($44 \times 6h = 264h$ low, $44 \times 12h = 528h$ high).

2. Use-Case Points: Looking at the actors and use cases, gave us 30 use-case points. At a standard rate of 20 hours per point, this method estimated 600 team hours. ($30 \times 20h = 600h$).

The difference between both estimates relies on the fact that the function points gave us a middle value of 352 hours, while use-case points resulted in 240 hours, creating an estimation gap of about 1.5 ($352 \div 240 = 1.467$). The two estimates are reasonably close, but the difference between both estimates comes down to scope perspective: function points focus strictly on counting discrete inputs, screens, and stored data, while use-case points look at the full user journey and factor in real-world friction like team coordination, setup, and testing edge cases.

With 264 hours on the low and 600 hours on the high end, the team is setting baseline target at 495 project hours across the team with an estimation of 1.8 to 4 weeks to complete this core slice, fitting into the 3-month total development plan.

Total labor cost is set at \$12,375 without overhead, up to \$15,000 at a \$25 / hour rate, staying within the budget range.

ROI Analysis

The initial investment to develop the application is estimated at \$12,375 for direct labor (495 hours at \$25/hour) and up to \$15,000 when accounting for tooling overhead, with zero infrastructure costs during Release 1 by relying on free hosting. In return, user interviews revealed that consumers routinely lose over \$100 over two years on small, forgotten subscriptions in the \$2 to \$10 range. By proactively notifying users before renewal dates and providing clear merchant cancellation paths, the app helps an active user eliminate between \$50 and \$120 each year in unwanted recurring fees. If a user saves an average of \$60 a year, it takes just 250 active users saving a combined \$15,000 in their first year to match our total project cost. This shows that even a small user base gets real value from the app while keeping our financial risk low.

Change Log

Change	Why	Downstream Effect
Scope is centered toward recurring subscription management, long term costs, upcoming charges, and cancellation guidance.	The interviews showed more problems than users forgetting subscriptions. The main pattern we saw was users overlooking small charges and experiencing difficult cancellation processes.	Features and estimations focus on recurring charge detection, notifications, cumulative cost information, and cancelation guidance.
Direct banking API's and automated third-party cancellation placed outside of project scope.	Features exceeded practical scope of current project / sprint schedule.	Development focuses on subscription tracking, cancellation guidance without requiring direct banking, or third-party cancellation integrations.
Currency conversion and automated push alerts relying on calendar checks were deferred.	Features not necessary for initial scope.	Features not included in current release and can be considered for future development.
Project effort established: 495-hour baseline.	Sprint 2 introduced formal estimation using function	495-hour baseline can be used for staffing, FTE month

	points and use case points to estimate project effort.	labor cost and budget calculations.
Project budget range was established	Converted estimated effort into labor cost using an assumed hourly rate.	Direct labor estimate: \$12,375 Total project cost estimate: \$15,000 (tooling/ overhead considered).